

def
statements

CSCI
134

but wait!

there's another way to
create new functions!

two ways to create a function

way 1: lambda expressions

name = **lambda** **arguments** : **return value**
(comma-separated) (what the function computes)

e.g.

```
double = lambda x : 2*x
```

way 2: def statements

```
def name (arguments) :  
    return value  
    (comma-separated) (what the function computes)
```

4 spaces

e.g.

```
def double(x):  
    return 2*x
```

two ways to create a function

way 1: lambda expressions

name = **lambda** **arguments** : **return value**
(comma-separated) (what the function computes)

e.g.

```
diff = lambda s1, s2 : abs(len(s1) - len(s2))
```

way 2: def statements

```
def name (arguments) :  
    return value  
    (comma-separated) (what the function computes)
```

4 spaces

e.g.

```
def diff(s1, s2):  
    return abs(len(s1) - len(s2))
```

why does python provide

two ways

to create a function?

way 1: lambda expressions

name = **lambda** **arguments** : **return value**
(comma-separated) (what the function computes)

way 2: def statements

```
def name (arguments) :  
    return value  
    (comma-separated) (what the function computes)
```

put as many lines as you like here

4 spaces

↑
advantage: you can write
multiline functions

```
def diff(s1, s2):  
    print("zomg i'm doing a diff")  
    relative_diff = len(s1) - len(s2)  
    return abs(relative_diff)
```

way 1: lambda expressions

name = **lambda** **arguments** : **return value**
(comma-separated) (what the function computes)

↑
advantage: quick
and disposable

way 2: def statements

```
def name (arguments):  
    return value
```

(comma-separated) (what the function computes)

put as many lines as you like here

4 spaces



def statements are used far
more frequently than lambda expressions
but both are useful

exercise :

Fill in the blanks to create a function that computes the reciprocal of a number

(the reciprocal of x is $\frac{1}{x}$)

```
In [1]: reciprocal = 
```

```
In [2]: reciprocal(4)
```

```
Out[2]: 0.25
```

```
In [3]: reciprocal(10)
```

```
Out[3]: 0.1
```

```
In [1]: def   
        ...:  
        ...:
```

```
In [2]: reciprocal(4)
```

```
Out[2]: 0.25
```

```
In [3]: reciprocal(10)
```

```
Out[3]: 0.1
```

exercise :

Fill in the blanks to create a function that computes the reciprocal of a number

(the reciprocal of x is $\frac{1}{x}$)

```
In [1]: reciprocal = lambda x: 1/x
```

```
In [2]: reciprocal(4)
```

```
Out[2]: 0.25
```

```
In [3]: reciprocal(10)
```

```
Out[3]: 0.1
```

```
In [1]: def reciprocal(x):  
...:     return 1/x  
...:
```

```
In [2]: reciprocal(4)
```

```
Out[2]: 0.25
```

```
In [3]: reciprocal(10)
```

```
Out[3]: 0.1
```